

EMCALI Cyber 2.0 Híbrido Enterprise v4

1. Contexto y evolución del proyecto

La versión Enterprise v4 representa la evolución de EMCALI Cyber 2.0 Híbrido v3 hacia una plataforma de

El sistema pasó de:

- configuración estática,
- modelos hardcoded,
- backend único,
- prompts rígidos,
- inferencia limitada,

a una arquitectura:

- híbrida,
- multi-backend,
- configurable dinámicamente,
- preparada para operación local y cloud,
- soportada por healthcheck,
- con monitoreo operacional.

La evolución fue motivada por la necesidad de soportar hardware heterogéneo, operación offline, resilien

2. Objetivos de la versión Enterprise

- Incorporar soporte multi-LLM.
- Permitir cambio dinámico de modelos.
- Detectar automáticamente modelos instalados.
- Integrar Ollama local.
- Incorporar fallback Gemini/OpenAI.
- Implementar healthcheck inteligente.
- Reducir dependencia de configuración hardcoded.
- Implementar configuración centralizada.
- Mejorar estabilidad de inferencia.
- Habilitar arquitectura cloud/on-premise.

3. Arquitectura Enterprise Actualizada

flowchart TB

A[Fuentes Vulnerabilidades]
--> B[Motor Ingesta]

B --> C[Normalizador]
C --> D[findings_df]

D --> E[Motor Riesgo]
D --> F[RAG Engine]

F --> G[Vector Store]

E --> H[Prompt Builder]

H --> I[LLM Orchestrator]

I --> J[Ollama Local]
I --> K[Gemini]
I --> L[OpenAI]
I --> M[Offline Engine]

I --> N[Healthcheck]

N --> O[Timeout]

```
N --> P[GPU/RAM]
N --> Q[Latency]

I --> R[Cache Inferencia]

I --> S[Async Queue]

I --> T[Streaming Response]

I --> U[Dashboard Streamlit]

U --> V[Informes]
U --> W[Métricas]
U --> X[Remediación]
```

4. Cambios principales implementados

4.1 Configuración dinámica

Antes:

- modelo fijo llama3.1:8b
- backend hardcoded
- timeout fijo
- parámetros estáticos

Ahora:

- selección dinámica de backend
- selección dinámica de modelo
- timeout configurable
- parámetros runtime
- soporte multi-modelo

4.2 Integración Ollama Enterprise

- detección automática mediante 'ollama list'
- selección dinámica desde UI
- soporte modelos livianos y pesados
- validación automática del puerto 11434
- fallback automático

4.3 Nuevos modelos soportados

- phi3:mini
- tinyllama
- gemma:2b
- mistral
- llama3
- modelos futuros Ollama

4.4 Configuración híbrida

La aplicación ahora soporta:

- local on-premise
- cloud IA
- operación híbrida
- operación offline

5. Sistema de inteligencia híbrida

El nuevo motor híbrido desacopla la lógica de inferencia del backend físico.

Características:

- abstracción de modelos
- routing dinámico
- fallback inteligente
- balanceo funcional
- independencia del proveedor IA

Backends soportados:

- Ollama
- Gemini
- OpenAI
- Offline Mock

Beneficios:

- resiliencia
- continuidad
- soberanía tecnológica
- flexibilidad operacional

6. Healthcheck y resiliencia

Se incorporó un sistema de validación automática de backend.

Funciones:

- verificación puerto Ollama
- validación timeout
- detección modelos instalados
- control de latencia
- fallback automático
- monitoreo de errores

Problemas corregidos:

- python313.dll
- hardcoding
- timeout Ollama
- errores de configuración
- fallbacks inválidos
- prompts sobredimensionados

7. Gestión avanzada de prompts

La aplicación implementa ahora:

- prompt builder desacoplado
- control de tamaño
- limitación de contexto
- soporte RAG contextual
- prompts dinámicos

Se redujo:

- consumo memoria
- latencia
- timeouts
- sobrecarga LLM

Especialmente en equipos:

- Dell T1650
- HP Z600
- equipos sin GPU dedicada

8. Sistema RAG Enterprise

El sistema RAG evolucionó desde un esquema estático hacia un sistema contextual enterprise.

Fuentes:

- playbooks
- CVEs
- NVD
- CISA KEV
- EPSS
- documentación interna
- arquitectura Foundation■Sec

Mejoras:

- chunking contextual
- recuperación optimizada
- evidencia transversal
- reutilización de contexto

9. Monitoreo operacional

Nuevos componentes:

- monitoreo GPU
- monitoreo RAM
- latencia inferencia
- backend activo
- timeout runtime
- estado backend

Beneficios:

- observabilidad
- troubleshooting
- tuning operacional
- optimización hardware

10. Arquitectura objetivo futura

flowchart LR

A[Tenable/OpenVAS]

--> B[Motor IA]

B --> C[Agentes Autónomos]

C --> D[SIEM]

C --> E[ITSM]

C --> F[SOC]

B --> G[Graph RAG]

G --> H[Memory Systems]

B --> I[LLM Local]

B --> J[LLM Cloud]

I --> K[Ollama]

J --> L[Gemini/OpenAI]

C --> M[Remediación Automática]

M --> N[Playbooks Autónomos]

11. Beneficios estratégicos para EMCALI

- modernización SOC
- IA aplicada a ciberseguridad
- soporte UAML
- autonomía tecnológica
- reducción dependencia cloud
- operación híbrida
- preparación SOC cognitivo
- resiliencia operacional
- soporte IT/OT

12. Roadmap

Fase 1:

- estabilización multi-**backend**
- healthcheck
- configuración dinámica

- Fase 2:
- integración SIEM
 - integración OpenVAS
 - integración Tenable

- Fase 3:
- agentes autónomos
 - graph RAG
 - copiloto SOC

- Fase 4:
- remediación automática
 - IA agentic
 - SOC cognitivo

13. Conclusiones

La nueva versión Enterprise transforma EMCALI Cyber 2.0 en una plataforma híbrida de ciberseguridad asis

- La solución ya soporta:
- multi-**LLM**,
 - inferencia híbrida,
 - RAG contextual,
 - operación on-**premise**,
 - configuración dinámica,
 - monitoreo operacional,
 - resiliencia cloud/local.

La arquitectura obtenida constituye una base sólida para evolucionar hacia plataformas cognitivas de def

Comparativo Arquitectónico

Componente	v3	Enterprise v4	Beneficio
Modelo	Hardcoded	Dinámico	Flexibilidad
Backend	Limitado	Multi- backend	Resiliencia
Ollama	Básico	Healthcheck + detección	Estabilidad
RAG	Simple	Enterprise contextual	Precisión
Prompt	Estático	Optimizado	Menor latencia
Monitoreo	No	GPU/RAM	Observabilidad
Fallback	Parcial	Inteligente	Continuidad
Configuración	Manual	Centralizada	Gobernabilidad